



Project Proposal

[Spectra.jl across the electromagnetic spectrum](#)

Mentors: [@fiebaker](#), [@icweaver](#), [@cgarling](#), [@barrettp](#)

Personal Information

Name: ADITYA KUMAR PANDEY

Email: vp220626@gmail.com

GitHub: [aditya-pandey-dev](#)

Slack: [Aditya Pandey](#)

Matrix: [@aditya-pandey-dev:matrix.org](#)

Time Zone: IST (GMT + 5:30)

TABLE OF CONTENT:

PR links: open source contributions	3
Contributions to the JuliaAstro	3
Pull requests in RADIS software	3
Pull requests in Astroquery / Astropy	4
Some of my pull requests in OpenAstronomy repos	4
My Background:	5
Interest in Open Astronomy:	6
1.0 Project Overview	7
Anticipated Scientific Impact	8
System Architecture Overview (Visual Aid)	9
Summary	9
2.0 Milestones (as mentioned in project idea)	11
3.0 Deliverables	12
3.1.1 Migrate OGIP Parser from SpectralFitting.jl	13
3.1.2 Implement Multi-Format Data Loading	17
3.1.3 Common Spectral Utilities	21
Risk Management and Mitigation Strategies	24
4.0 Pre-community bonding and current working:	25
5.0 GSOC timeline and my availability	25
5.2 Availability	27
6.0 Some of my projects	27
7.0 What I am studying and for what	27
8.0 Appreciation:	29
9.0 Gsoc:	29

PR links: open source contributions

Pull requests in JuliaAstro

1. [SpectralFitting.jl PR #241](#): (closes #196, ref #132) Multi-dataset evaluator with domain union utilities — Implemented support for evaluating spectral models across multiple datasets simultaneously
2. [SpectralFitting.jl PR #242](#): (closes #187) Add XSPEC model string macro — Added macro support for parsing XSPEC-style model definition strings in Julia
3. [JuliaAstro.github.io PR #203](#): Add multi-wavelength spectroscopy tutorial — Contributed documentation tutorial for multi-wavelength spectroscopic analysis on the JuliaAstro website

Contributions to the RADIS software

4. [PR #894](#): PR #894: (Issue #34) Legacy RADIS database management improvement — Refactored database I/O operations for better HDF5 file handling and caching performance
5. [PR #981](#): PR #981: (ref #978, #658) Add Polars/Parquet lazy-loading backend with DataFrameAdapter — Implemented the core DataFrameAdapter abstraction layer and PolarsAdapter backend for replacing Vaex with modern lazy-loading
6. [PR #971](#): (Issue #602) Include other broadening partners than air for ExoMol — Added support for multiple broadening species (H2, He, CO2) beyond default air broadening in ExoMol database calculations
7. [PR #924](#): PR #924: (Issue #268) Code improvement and feature enhancement — Addressed spectroscopic computation optimization in the RADIS calculation pipeline
8. [PR #958](#): PR #958: (Issue #474) Database management improvement — Enhanced DataFileManager operations for better handling of cached spectroscopic databases

Pull requests in Astroquery / Astropy

9. [Astroquery PR #3536](#): Astroquery PR #3536: (Issue #925) Astroquery integration — Contributed to the astronomical data querying package for improved RADIS-Astropy interoperability
10. [Astropy Issue #19345](#): Astropy Issue #19345: Identified and reported a bug in the Astropy core repository, currently working on the fix

Some of my pull requests in other open source projects<OpenAstronomy>

11. [github-actions-workflows PR #341](#): Contributed to OpenAstronomy's shared CI/CD GitHub Actions workflow repository, improving automation for all sub-organizations.

My Background:

I am a 6th-semester BTech student specializing in Artificial Intelligence & Machine Learning at Shri Shankaracharya Institute of Professional Management & Technology (SSIPMT), Raipur, India. I have been working with Python, JavaScript, Java, and have strong experience in data science and scientific computing using NumPy, Pandas, Matplotlib, TensorFlow, and Keras. I also work with full-stack technologies like React, Next.js, MongoDB, and Django.

Beyond academics, I have a deep fascination with scientific computing, spectroscopy, and high-performance data processing. I have explored these topics through my contributions to the RADIS project under OpenAstronomy and SpectralFitting.jl under JuliaAstro, where I gained hands-on experience with spectroscopic data formats, FITS file handling, and X-ray spectral analysis workflows.

I bring a unique combination of skills directly relevant to this project: - SpectralFitting.jl Contributions: PRs #241 and #242 gave me hands-on experience with OGIP data structures (OGIPDataset, ResponseMatrix) that need to be migrated to [Spectra.jl](#), OGIP Format Knowledge: I understand PHA/RMF/ARF file structure and how X-ray spectral data differs from optical wavelength-based spectra., JuliaAstro Ecosystem: My multi-wavelength tutorial (PR #203) shows familiarity with Spectra.jl, SpectralFitting.jl, FITSIO.jl integration.

Key Achievements:

- Winner of college-level hackathon "Navonmesh" – demonstrated ability to rapidly prototype and build solutions under time pressure
- Selected for Smart India Hackathon (SIH) national level – Top 25 teams out of thousands of participants nationwide, showcasing strong problem-solving skills
- 7th rank in Indian National Physics Olympiad at national level (Class 12th) – demonstrates strong foundation in physics and scientific reasoning, directly relevant to spectroscopic computing

Interest in Open Astronomy:

I am excited about Open Astronomy's mission to advance open-source tools for astrophysical research. The opportunity to contribute to Spectra.jl aligns perfectly with my growing interest in scientific computing.

I deeply value Open Astronomy's commitment to **open science, reproducibility, and collaboration**. This project presents an opportunity to apply my skills, learn from experts, and contribute to tools that push the boundaries of **high-resolution spectroscopy**. I look forward to being part of a community driving innovation in astronomical data analysis.

1.0 Project Overview

Spectra.jl is a Julia package designed to simplify the treatment of spectroscopic data. It provides straightforward tools for common tasks such as loading spectra, applying transformations, and visualization, while preserving the flexibility and power of the Julia ecosystem.

Spectroscopic analysis plays a critical role in modern astrophysics. By studying the distribution of light across wavelengths, researchers can determine the chemical composition, temperature, density, and velocity of astronomical objects. From detecting exoplanet atmospheres using JWST infrared spectra to understanding accretion physics through Chandra X-ray observations, spectroscopy is fundamental to answering key questions about the universe.

However, the current Spectra.jl implementation focuses primarily on optical spectroscopy with limited support for other wavelength regimes. To facilitate multi-wavelength research, this project will extend Spectra.jl to support spectroscopic data from across the entire electromagnetic spectrum. The core enhancements will include:

OGIP Parser Migration — Moving the X-ray spectral data loader (PHA/RMF/ARF files) from SpectralFitting.jl into Spectra.jl, making X-ray data accessible without requiring the full fitting infrastructure.

Multi-Format Data Loading — Implementing loaders for at least three spectral formats: OGIP (X-ray), SDSS (optical), and HST (UV), with automatic format detection from FITS headers.

Common Spectral Utilities — Building universal routines for rebinning (by minimum counts, signal-to-noise, or fixed bins), wavelength-energy conversion ($E = hc/\lambda$), and redshift correction that work across all spectral types.

By leveraging Julia's speed and multiple dispatch system, Spectra.jl aims to provide a unified interface for spectral data handling. This approach will enable researchers to seamlessly combine X-ray, UV, optical, and infrared observations in a single analysis workflow, providing astronomers with a robust and optimized framework for multi-wavelength spectroscopy.

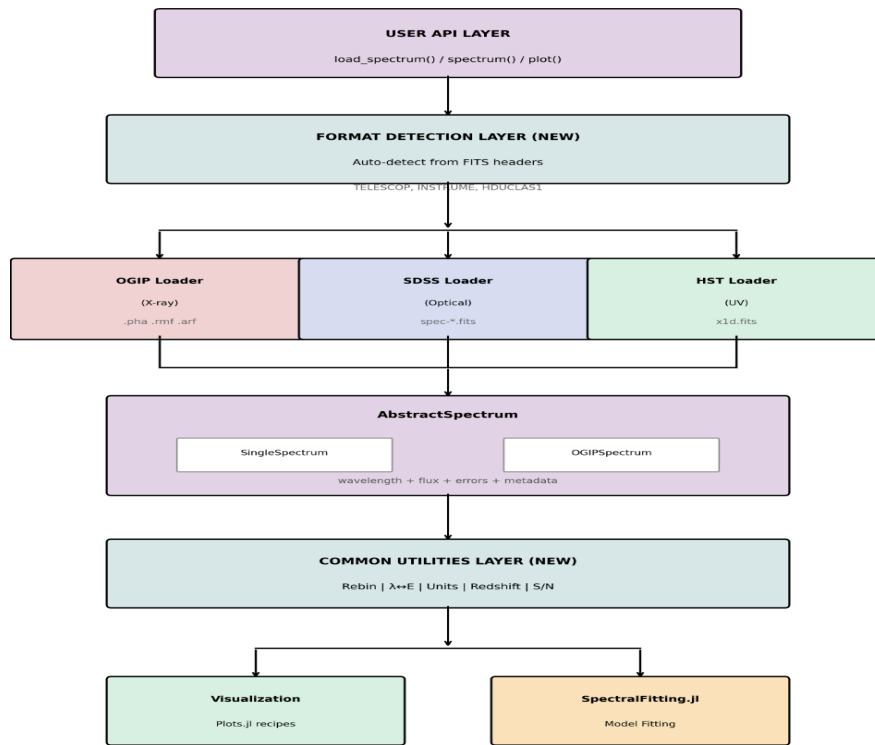
Anticipated Scientific Impact

This project has the potential to drive significant advances in multi-wavelength astrophysics by enabling unified spectral analysis tools. The improved accessibility of the Julia-based tools may help answer key scientific questions such as:

- **Multi-Wavelength AGN Studies:** How does the X-ray corona emission from active galactic nuclei relate to the optical/UV accretion disk spectrum? With unified loading, researchers can directly compare Chandra X-ray and HST UV spectra in a single workflow.
- **Exoplanet Atmospheric Characterization:** What molecular species can we detect in JWST transmission spectra? Consistent rebinning and unit conversion utilities will simplify analysis pipelines for the exoplanet community.
- **Stellar Wind Analysis:** How do stellar outflows manifest differently in UV absorption lines versus X-ray emission? Unified spectral types make cross-wavelength comparison straightforward.
- **Educational Accessibility:** By providing a single, well-documented package that handles all spectral formats, we lower the barrier for students and researchers new to spectroscopy. They can focus on science rather than data format wrangling.

By offering a computational platform with unified data handling across the electromagnetic spectrum, the project is expected to facilitate rapid prototyping, testing, and validation of multi-wavelength analyses, ultimately accelerating discoveries in the field.

System Architecture Overview (Visual Aid)



#This diagram was generated using matplotlib

Summary

The primary goal of this project is to extend Spectra.jl to become the central package for spectral data handling in the JuliaAstro ecosystem. By migrating the OGIP parser from SpectralFitting.jl and implementing support for multiple file formats (X-ray, optical, UV), Spectra.jl will offer features that are currently fragmented across different packages. The addition of common utilities (rebinning, wavelength-energy conversion, redshift correction) will provide astronomers with a unified and optimized framework for multi-wavelength spectroscopic analysis, facilitating groundbreaking discoveries across the electromagnetic spectrum.

Technical Implementation Roadmap:

A. OGIP Parser Migration (from SpectralFitting.jl)

- Extract PHA, RMF, ARF file parsing functions
- Create OGIPSpectrum <: AbstractSpectrum type
- Implement ResponseMatrix type for instrument response
- Ensure SpectralFitting.jl can depend on Spectra.jl for data loading

B. Format Auto-Detection System

- Read FITS header keywords (HDUCLAS1, TELESCOP, INSTRUME)
- Automatically identify file format (OGIP, SDSS, HST, generic)
- Provide unified load_spectrum() interface

C. Multi-Format Loaders

- OGIP loader for X-ray spectra (channels, counts, response matrix)
- SDSS loader for optical spectra (wavelength from loglam, flux, ivar)
- HST loader for UV spectra (x1d.fits from COS/STIS instruments)

D. Common Utilities

- Rebinning: by minimum counts, signal-to-noise ratio, or fixed bins
- Wavelength $\leftrightarrow$ Energy conversion: $E(\text{keV}) = 12.398 / \lambda(\text{\AA})$
- Redshift correction: $\lambda_{\text{rest}} = \lambda_{\text{obs}} / (1 + z)$
- Signal-to-noise calculation per channel

E. Testing & Documentation

- Unit tests for all loaders and utilities
- Integration tests with real astronomical data
- Docstrings and tutorial notebooks

2.0 Milestones{as mentioned in project idea}

☒ **Coding starts:**

☒ **Learn about different spectral data types and what makes radio or X-ray spectra unusual**

I have gained familiarity with spectral data types by implementing operations and making contributions to SpectralFitting.jl [[my_prs](#)]. I understand that X-ray spectra use energy axis (keV) with counts per channel, requiring response matrices for energy calibration.

☒ **Learn about FITS tables and other common astronomical data formats for spectra**

I have applied this knowledge on existing codebases. Through SpectralFitting.jl and RADIS work, I understand OGIP standard (PHA/RMF/ARF files), SDSS spectrum format, and HST x1d format. My [notebook] carries examples of how I have read and processed FITS spectral files.

☒ **Learn about Test Driven Development (TDD), and techniques for writing robust and reliable software**

A series of tests is part of my contributions. PR #241 includes comprehensive tests for multi-dataset evaluation, and @fjebaker's feedback helped me improve test coverage and code quality in my PRs.

☒ **Familiarise yourself with existing routines in Spectra.jl, and the OGIP parser in SpectralFitting.jl**

I have studied both codebases: Spectra.jl's `AbstractSpectrum` type hierarchy and SpectralFitting.jl's OGIP parser in [src/datasets/ogip.jl](#). My PRs #241 and #242 demonstrate hands-on familiarity with the codebase.

☐ **1st evaluation**

☒ **Move and integrate the OGIP parser from SpectralFitting.jl to Spectra.jl**

The OGIP parser migration is the primary deliverable. I will extract `OGIPDataset` and `ResponseMatrix` types, create `OGIPSpectrum <: AbstractSpectrum` in `Spectra.jl`, and ensure `SpectralFitting.jl` can depend on `Spectra.jl` for data loading.

☒ **Implement data loading for at least one spectral datatype of choice**

I will implement SDSS optical spectrum loader (spec-*.fits files) with format auto-detection based on FITS header keywords (`TELESCOP`, `INSTRUME`).

☐ **Final evaluation**

☐ **A revamped `Spectra.jl`; that is:**

☐ **Data loading for at least three different file formats**

- OGIP (X-ray) — PHA/RMF/ARF files
- SDSS (Optical) — spec-*.fits files
- HST (UV) — x1d.fits from COS/STIS

☐ **Common routines for spectral analysis**

- Rebinning by minimum counts, signal-to-noise, or fixed bins
- Wavelength ↔ Energy conversion: $E(\text{keV}) = 12.398 / \lambda(\text{\AA})$
- Redshift correction: $\lambda_{\text{rest}} = \lambda_{\text{obs}} / (1 + z)$
- Signal-to-noise calculation per channel

3.0 Deliverables

1. Core Format Support Modules:

- Migrate OGIP parser from `SpectralFitting.jl` to `Spectra.jl`
- Implement data loading for SDSS optical spectra
- Implement data loading for HST UV spectra

2. I/O and Testing Infrastructure: [as I will develop and extend functionalities, I will implement edge cases in Tests.]

- Develop robust I/O operations for FITS files
- This is the [HEASARC archive](#) for X-ray data. I will download test spectra from here and use this data for testing.
- Create comprehensive test suites ensuring code reliability and performance.

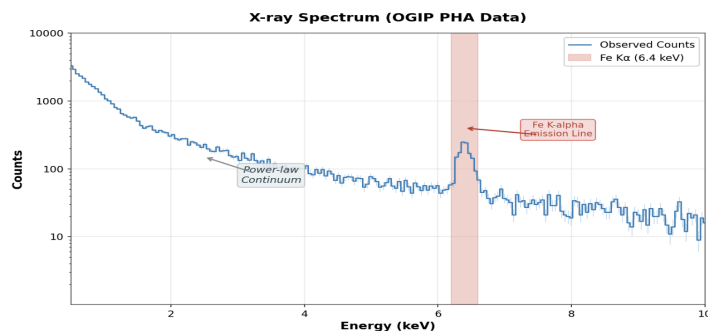
3. Common Spectral Utilities:

- Implement rebinning functions (by counts, S/N, fixed bins)
- Add wavelength $\leftrightarrow$ energy conversion system
- Implement redshift correction utilities

3.1.1 Migrate OGIP Parser from SpectralFitting.jl

What is OGIP?

OGIP (Office of Guest Investigator Programs) is a standard developed by NASA's HEASARC for storing X-ray spectral data. Unlike optical spectra which directly store flux vs wavelength, X-ray spectra require a more complex representation because of how X-ray detectors work.



Why X-ray Spectra are Different

When an X-ray photon hits a CCD detector:

1. The photon creates a charge cloud in the detector
2. This charge is measured as a "pulse height" in a channel number
3. The relationship between channel and true photon energy is **probabilistic**
4. This probability is captured in the **Response Matrix (RMF)**

This means X-ray observations require three files working together:

File	Extension	Contains	Purpose
PHA	.pha	Counts per channel	Raw observed data
RMF	.rmf	Response Matrix + EBOUNDS	Channel -> Energy mapping
ARF	.arf	Effective Area vs Energy	Telescope efficiency

The Problem

Currently, the OGIP parser exists in SpectralFitting.jl under [src/datasets/ogip.jl](#). But this code is tightly coupled to the fitting infrastructure. A user who simply wants to:

- Visualize an X-ray spectrum
- Convert X-ray energies to wavelengths
- Compare X-ray with optical data

...must install the entire SpectralFitting.jl package with all its optimization dependencies, just to read the files.

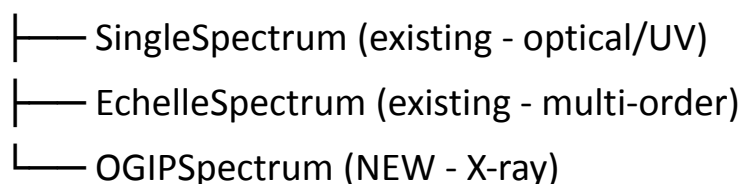
My Solution

I will extract the core OGIP reading functionality into Spectra.jl with clean separation:

- **Spectra.jl** -> handles data loading and basic manipulation
- **SpectralFitting.jl** -> handles model fitting (depends on [Spectra.jl](#))

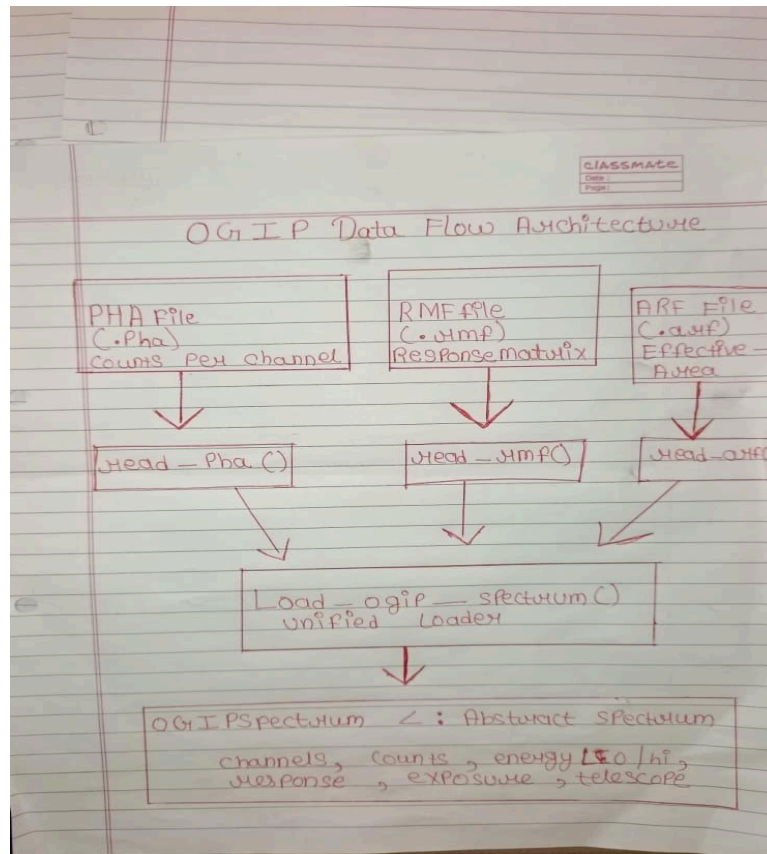
New Type Hierarchy:

AbstractSpectrum (existing in Spectra.jl)



Implementation Details

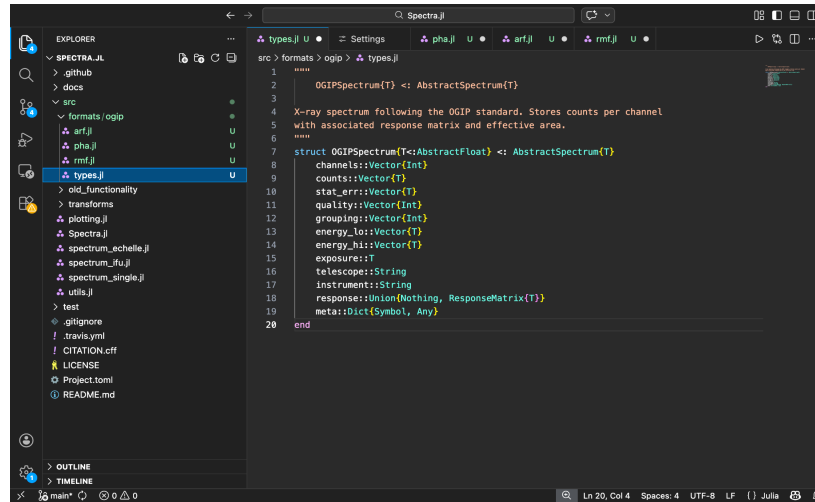
For this, I have decided to use an abstract type structure, as discussed in this [discussion](#) with @fjebaker.



The **OGIPSpectrum** type will store:

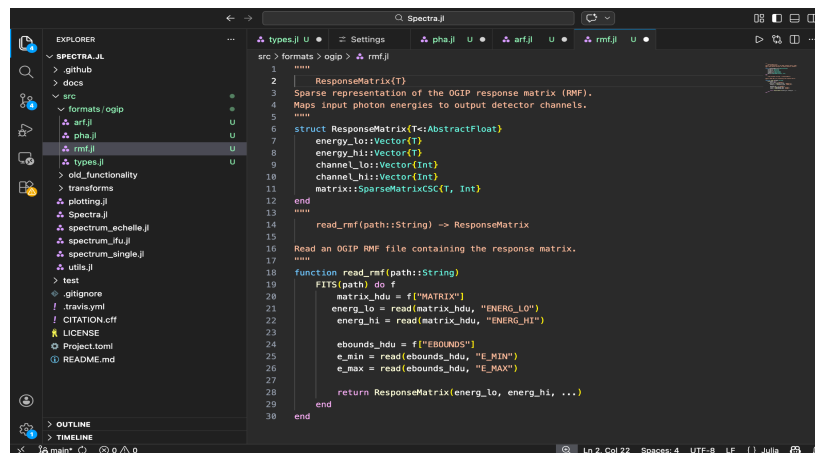
- **channels** - Channel numbers from detector
- **counts** - Observed counts per channel
- **energy_lo/hi** - Energy bounds for each channel (from RMF EBOUNDS)
- **exposure** - Integration time in seconds
- **response** - Optional ResponseMatrix for spectral analysis
- **telescope/instrument** - Metadata for identification

OGIPSpectrum Type Definition:-



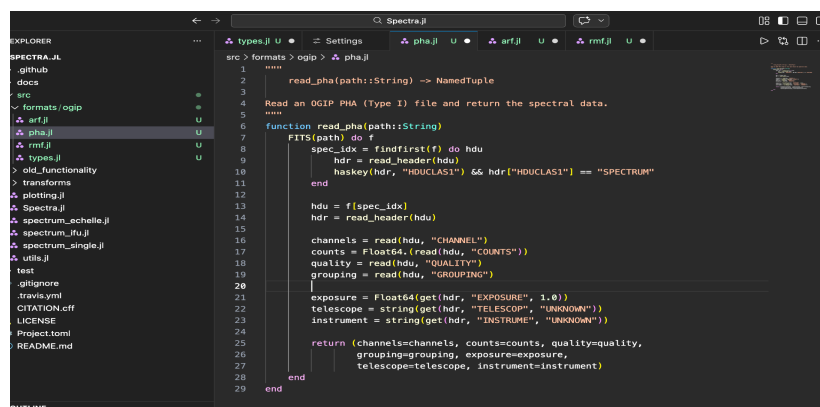
```
1  """
2  OGIPSpectrum{T} <: AbstractSpectrum{T}
3  """
4  X-ray spectrum following the OGIP standard. Stores counts per channel
5  with associated response matrix and effective area.
6  """
7  struct OGIPSpectrum{T<:AbstractFloat} <: AbstractSpectrum{T}
8      channels::Vector{Int}
9      counts::Vector{T}
10     stat_err::Vector{T}
11     quality::Vector{Int}
12     grouping::Vector{Int}
13     energy_lo::Vector{T}
14     energy_hi::Vector{T}
15     exposure::T
16     telescope::String
17     instrument::String
18     response::Union{Nothing, ResponseMatrix{T}}
19     meta::Dict{Symbol, Any}
20 end
```

RMF File Reader:-



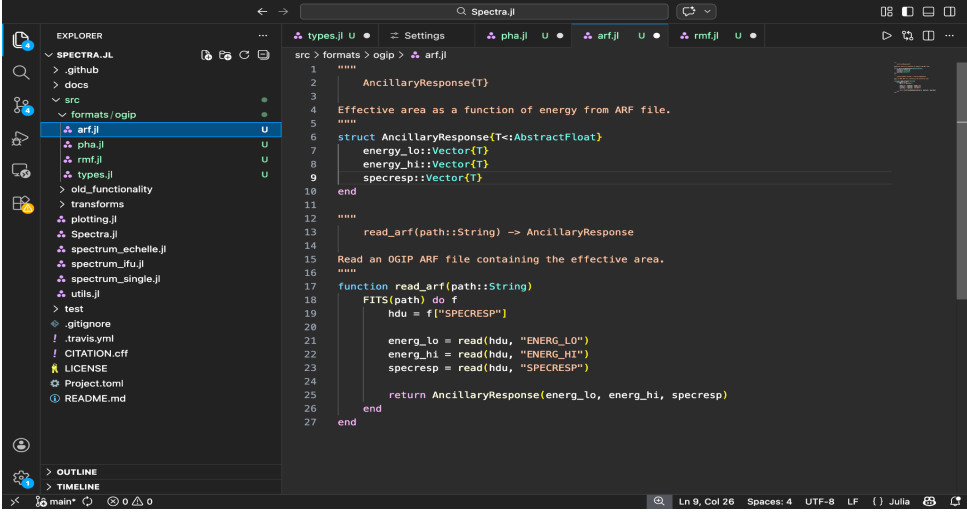
```
1  """
2  ResponseMatrix{T}
3  Sparse representation of the OGIP response matrix (RMF).
4  Maps input photon energies to output detector channels.
5  """
6  struct ResponseMatrix{T<:AbstractFloat}
7      energy_lo::Vector{T}
8      energy_hi::Vector{T}
9      channel_lo::Vector{Int}
10     channel_hi::Vector{Int}
11     matrix::SparseMatrixCSC{T, Int}
12 end
13 """
14 read_rmf(path::String) -> ResponseMatrix
15 """
16 Read an OGIP RMF file containing the response matrix.
17 """
18 function read_rmf(path::String)
19     FITS(path) do f
20         matrix_hdu = f["MATRIX"]
21         energ_lo = read(matrix_hdu, "ENERG_LO")
22         energ_hi = read(matrix_hdu, "ENERG_HI")
23
24         ebounds_hdu = f["EBOUNDS"]
25         e_min = read(ebounds_hdu, "E_MIN")
26         e_max = read(ebounds_hdu, "E_MAX")
27
28         return ResponseMatrix(energ_lo, energ_hi, ...)
29     end
30 end
```

PHA File Reader:-



```
1  """
2  read_pha(path::String) -> NamedTuple
3  """
4  Read an OGIP PHA (Type I) file and return the spectral data.
5  """
6  function read_pha(path::String)
7      FITS(path) do f
8          spec_idx = findfirst(f) do hdu
9              hdr = read_header(hdu)
10             haskey(hdr, "HDOCLASS1") && hdr["HDOCLASS1"] == "SPECTRUM"
11          end
12
13          hdu = f[spec_idx]
14          hdr = read_header(hdu)
15
16          channels = read(hdu, "CHANNEL")
17          counts = Float64.(read(hdu, "COUNTS"))
18          quality = read(hdu, "QUALITY")
19          grouping = read(hdu, "GROUPING")
20
21          exposure = Float64(get(hdr, "EXPOSURE", 1.0))
22          telescope = string(get(hdr, "TELESCOP", "UNKNOWN"))
23          instrument = string(get(hdr, "INSTRUME", "UNKNOWN"))
24
25          return (channels=channels, counts=counts, quality=quality,
26                  grouping=grouping, exposure=exposure,
27                  telescope=telescope, instrument=instrument)
28      end
29 end
```

ARF File Reader:-

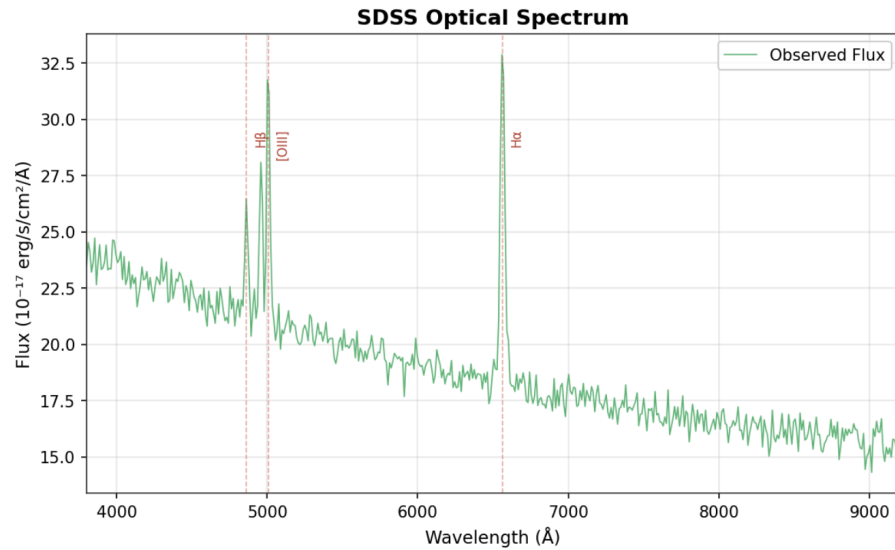


3.1.2 Implement Multi-Format Data Loading

Background: Why Multiple Formats?

Astronomical spectra come from telescopes observing at different wavelengths, and each has developed its own file format conventions:

Wavelength Regime	Example Instruments	File Format	Key Characteristics
X-ray (0.1-100 keV)	Chandra, XMM-Newton	OGIP PHA/RMF/ARF	Counts per channel, needs response matrix
UV (10-300 nm)	HST/COS, HST/STIS	x1d FITS	1D extracted spectra, wavelength calibrated
Optical (300-1000 nm)	SDSS, VLT/MUSE	Various FITS	Direct flux vs wavelength
Infrared (1-1000 μm)	JWST, Spitzer	x1d/s1d FITS	Similar to UV, different wavelength range

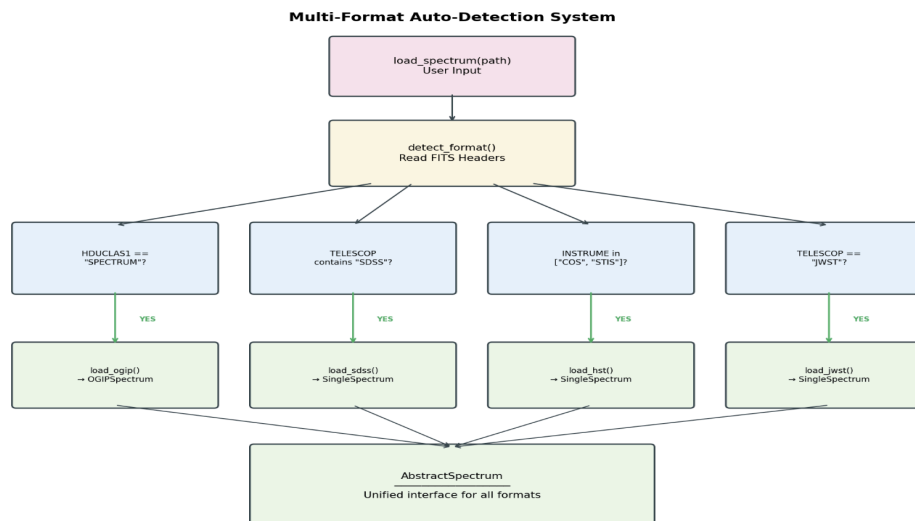


The Challenge

A researcher studying a multi-wavelength phenomenon (like an AGN) needs data from multiple wavelength regimes. Currently, they must write custom code for each format. **Spectra.jl** aims to provide a unified **load_spectrum()** interface.

Format Detection Strategy

The key insight is that FITS headers contain keywords that identify the file type:



FITS Header Keyword	Value	Detected Format
HDUCLAS1	"SPECTRUM"	OGIP X-ray
TELESCOP	contains "SDSS"	SDSS Optical
INSTRUME	"COS" or "STIS"	HST UV
TELESCOP	"JWST"	JWST IR

SDSS Optical Spectrum Loader

SDSS (Sloan Digital Sky Survey) spectra are stored in a specific FITS format with wavelength stored as $\log_{10}(\lambda)$. The loader will:

1. Parse `spec-*.fits` files
2. Convert `loglam` to linear wavelength
3. Extract flux and inverse variance
4. Return `SingleSpectrum` type

```

1  src > formats > sds.jl
2  =====
3  Load SDSS optical spectrum from spec-*.fits file.
4  =====
5  function load_sds_spectrum(path::String)
6      FITS(path) do f
7          hdu = f[2]
8
9          loglam = read(hdu, "LOGAM")
10         flux = read(hdu, "FLUX")
11         ivar = read(hdu, "IVAR")
12
13         wavelength = 10.0 .^ loglam
14
15         flux_err = 1.0 ./ sqrt.(ivar)
16         flux_err[isinf.(flux_err)] .= NaN
17
18         hdr = read_header(f[1])
19         return spectrum(
20             wavelength,
21             flux,
22             flux_err,
23             meta = Dict{
24                 :telescope => "SDSS",
25                 :plate => get(hdr, "PLATEID", 0),
26                 :id => get(hdr, "OBJID", 0),
27                 :fiber => get(hdr, "FIBERID", 0)
28             }
29         )
30     end
31 end

```

HST UV Spectrum Loader:-

```

1  src > formats > hst.jl
2  =====
3  Load HST UV spectrum from COS/STIS x1d.fits file.
4  =====
5  function load_hst_spectrum(path::String)
6      FITS(path) do f
7          hdu = f[2]
8
9          wavelength = read(hdu, "WAVELENGTH")
10         flux = read(hdu, "FLUX")
11         flux_err = read(hdu, "ERROR")
12
13         hdr = read_header(f[1])
14         return spectrum(
15             wavelength,
16             flux,
17             flux_err,
18             meta = Dict{
19                 :telescope => "HST",
20                 :instrument => get(hdr, "INSTRUME", "UNKNOWN"),
21                 :detector => get(hdr, "DETECTOR", "UNKNOWN"),
22                 :targetname => get(hdr, "TARGETNAME", "UNKNOWN")
23             }
24         )
25     end
26 end

```

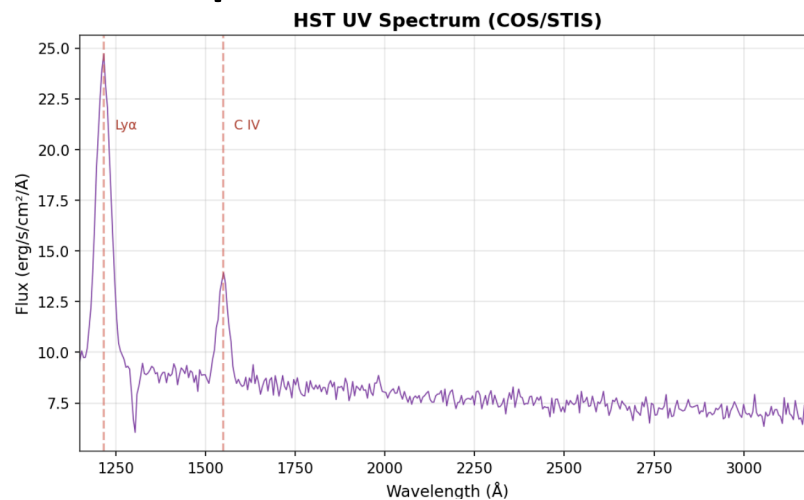
Format Auto-Detection System:-

```

src > formats > detection.jl
1
2 detect_format(path::String) -> Symbol
3 Auto-detect spectrum file format from FITS headers.
4
5 function detect_format(path::String)::Symbol
6     FITS(path) do f
7         hdr = read_header(f{1})
8
9         if haskey(hdr, "HDUCLASS1")
10             if hdr["HDUCLASS1"] == "SPECTRUM"
11                 return logip
12             end
13         end
14
15         if haskey(hdr, "TELESCOP")
16             if contains(uppercase(hdr["TELESCOP"]), "SDSS")
17                 return sdss
18             end
19         end
20
21         if haskey(hdr, "INSTRUME")
22             instr = uppercase(hdr["INSTRUME"])
23             if instr in ["COS", "STIS"]
24                 return hst_uv
25             end
26         end
27         return igeneric_fits
28     end
29 end
30
31
32
src > formats > detection.jl
1
2 function detect_format(path::String)::Symbol
3     FITS(path) do f
4         if haskey(hdr, "INSTRUME")
5             return igeneric_fits
6         end
7     end
8 end
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49

```

HST UV Spectrum:-

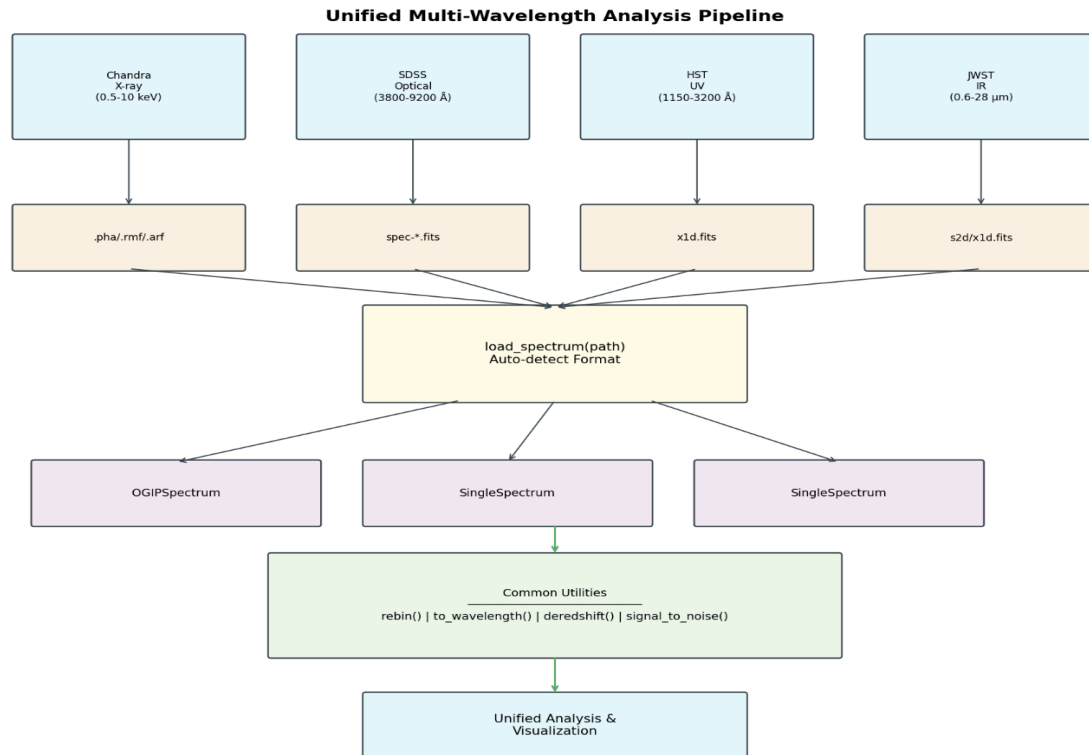


HST x1d files from COS/STIS instruments store extracted 1D spectra in the UV range (1150-3200 Å). The loader handles the specific FITS structure used by these instruments.

3.1.3 Common Spectral Utilities

Overview

These utilities will work across ALL spectrum types using Julia's multiple dispatch. This is one of the key advantages of having a unified **AbstractSpectrum** type hierarchy.

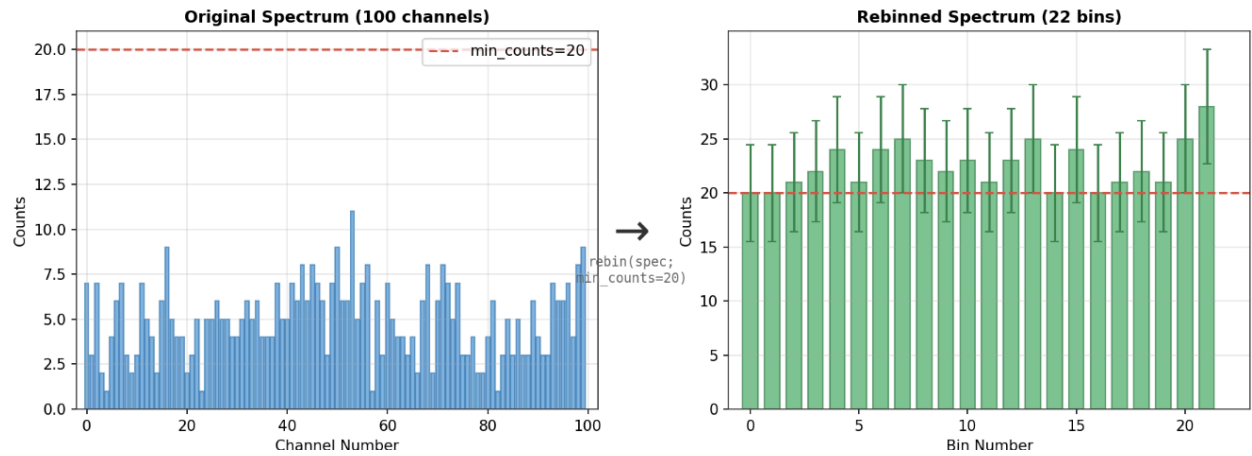


The basic steps are:

1. **Rebinning Functions:** Group channels based on different criteria
2. **Wavelength ↔ Energy Conversion:** Using $E = hc/\lambda$
3. **Redshift Correction:** Convert between observed and rest-frame
4. **Signal-to-Noise Calculation:** Per-channel and integrated S/N

Rebinning Implementation

X-ray spectra often have low counts per channel, making statistical analysis difficult. Rebinning groups adjacent channels to achieve a minimum count threshold.



Rebinning by Minimum Counts:-

```

src > utils > rebinning.jl
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32

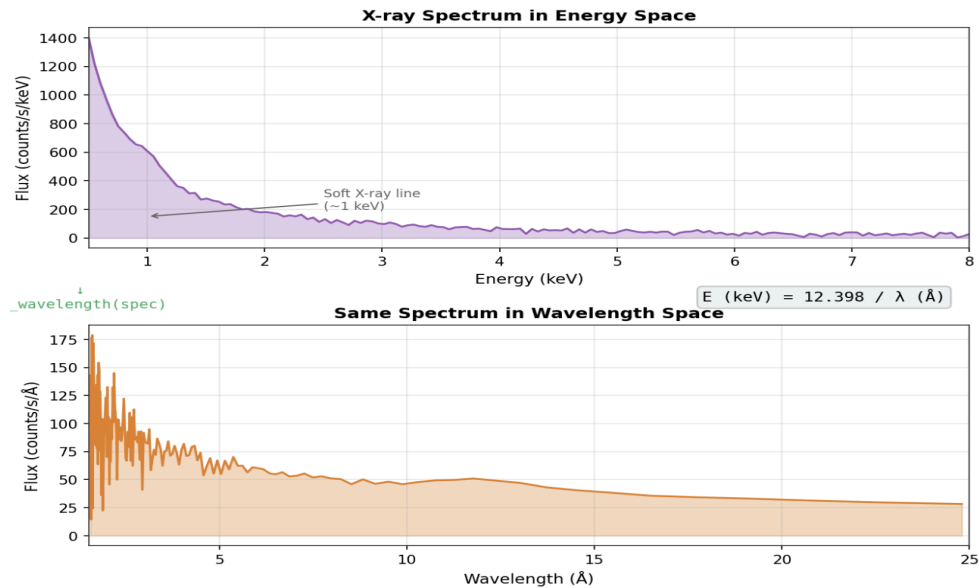
```

```

src > utils > rebinning.jl
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40

```

Wavelength ↔ Energy Conversion:-



The conversion between wavelength (Å) and energy (keV) uses:

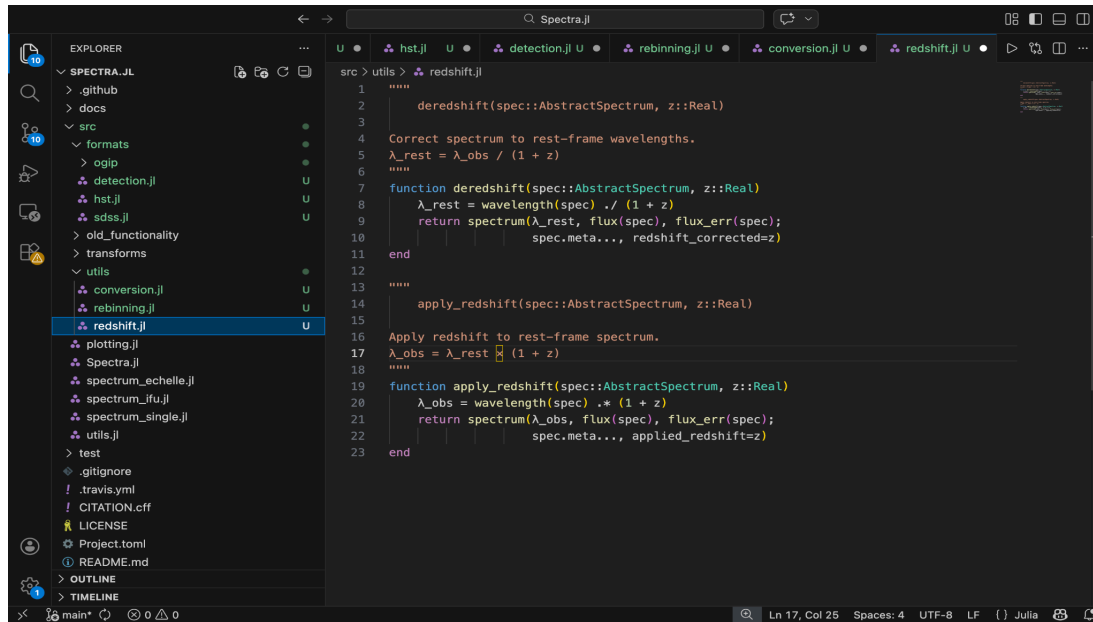
$$E \text{ (keV)} = 12.39842 / \lambda \text{ (Å)}$$

Wavelength-Energy Conversion:-

```
EXPLORER
├── SPECTRA.JL
│   ├── .github
│   ├── docs
│   ├── src
│   │   ├── formats
│   │   │   ├── ogip
│   │   │   ├── detection.jl
│   │   │   ├── hst.jl
│   │   │   ├── sdss.jl
│   │   ├── old_functionality
│   │   ├── transforms
│   │   └── utils
│   │       ├── conversion.jl
│   │       ├── rebinning.jl
│   │       ├── redshift.jl
│   │       ├── plotting.jl
│   │       ├── Spectra.jl
│   │       ├── spectrum_echelle.jl
│   │       ├── spectrum_ifu.jl
│   │       ├── spectrum_single.jl
│   │       ├── utils.jl
│   │       └── test
│   ├── .gitignore
│   ├── .travis.yml
│   ├── CITATION.cff
│   ├── LICENSE
│   ├── Project.toml
│   └── README.md
├── OUTLINE
└── TIMELINE

src > utils > conversion.jl
1  const hc_keV_Å = 12.39841984
2
3  wavelength_to_energy(λ_Å::Real) -> keV
4
5  Convert wavelength in Angstroms to energy in keV.
6  """
7  wavelength_to_energy(λ_Å::Real) = hc_keV_Å / λ_Å
8
9  """
10 energy_to_wavelength(E_keV::Real) -> Å
11
12 Convert energy in keV to wavelength in Angstroms.
13 """
14 energy_to_wavelength(E_keV::Real) = hc_keV_Å / E_keV
15
16 """
17 to_wavelength(spec::OGIPSpectrum)
18
19 Convert X-ray spectrum from energy to wavelength space.
20 """
21 function to_wavelength(spec::OGIPSpectrum)
22     λ_lo = energy_to_wavelength.(spec.energy_hi)
23     λ_hi = energy_to_wavelength.(spec.energy_lo)
24
25     E_mid = (spec.energy_lo + spec.energy_hi) ./ 2
26     flux_λ = flux(spec) .* E_mid.^2 ./ hc_keV_Å
27
28     return SingleSpectrum(λ_lo, λ_hi, flux_λ)
29 end
```

Redshift Correction:-



```
src > utils > redshift.jl
1  """
2  deredshift(spec::AbstractSpectrum, z::Real)
3
4  Correct spectrum to rest-frame wavelengths.
5  λ_rest = λ_obs / (1 + z)
6  """
7  function deredshift(spec::AbstractSpectrum, z::Real)
8      λ_rest = wavelength(spec) ./ (1 + z)
9      return spectrum(λ_rest, flux(spec), flux_err(spec);
10                     spec.meta..., redshift_corrected=z)
11  end
12
13  """
14  apply_redshift(spec::AbstractSpectrum, z::Real)
15
16  Apply redshift to rest-frame spectrum.
17  λ_obs = λ_rest * (1 + z)
18  """
19  function apply_redshift(spec::AbstractSpectrum, z::Real)
20      λ_obs = wavelength(spec) .* (1 + z)
21      return spectrum(λ_obs, flux(spec), flux_err(spec);
22                     spec.meta..., applied_redshift=z)
23  end
```

Risk Management and Mitigation Strategies:-

As with any software project, several risks may arise. Below are identified risks with proposed strategies to mitigate them:

1. FITS Format Variations:

- **Risk:** Different observatories may have slight variations in FITS file structure.
- **Mitigation:** Test with real data from multiple sources (Chandra, SDSS, HST) and implement robust error handling.

2. Backward Compatibility with SpectralFitting.jl:

- **Risk:** After migrating OGIP parser to Spectra.jl, SpectralFitting.jl may break.
- **Mitigation:** Ensure SpectralFitting.jl can depend on Spectra.jl for data loading, and maintain existing test coverage.

3. Integration with Existing Code:

- **Risk:** New OGIP types may conflict with existing AbstractSpectrum interface.
- **Mitigation:** Follow existing code patterns in Spectra.jl and write comprehensive tests before merging.

4.0 Pre-community bonding and current working:

I have joined the Slack channels for Julia, Stingray, and the Matrix channel for OpenAstronomy, where I introduced myself and began engaging with the community. While studying the project repositories, I familiarized myself with the codebase structure and existing implementations.

During this process, I studied the Spectra.jl repository and understood the AbstractSpectrum type hierarchy. I also explored the SpectralFitting.jl package, specifically the OGIP parser implementation in [src/datasets/ogip.jl](#), which will be migrated to Spectra.jl as part of this project.

This understanding of the existing codebase remains part of my preparation to ensure smooth integration of new format loaders with the existing infrastructure.

What I am currently doing:

- Studying the Spectra.jl codebase and understanding how spectrum types are structured
- Exploring the OGIP standard (PHA, RMF, ARF file formats) from HEASARC documentation
- Understanding how FITSIO.jl works for reading astronomical data files
- Reviewing the related discussion in [PR #24](#) to understand project direction

5.0 GSOC timeline and my availability

Period	Tasks
Apr 30 - May 25(Community Bonding)	1. Connect with mentors (@fjebaker, @icweaver, @cgarling, @barrettp) and understand project expectations and coding conventions. 2. Learn about different spectral data types (radio, X-ray, optical, UV) and what makes each unique in terms of data structure. 3. Study FITS tables, OGIP standard (PHA, RMF, ARF files), and familiarize with the OGIP parser in SpectralFitting.jl. 4. Learn about Test Driven Development (TDD) and techniques for writing robust and reliable software

	.5. Explore Spectra.jl codebase, understand AbstractSpectrum type hierarchy, and finalize milestones with mentors.
May 26 - Jun 16(Coding Phase 1)	1. Migrate OGIP parser from SpectralFitting.jl to Spectra.jl, implementing OGIPSpectrum type that extends AbstractSpectrum. 2. Create read_pha(), read_rmf(), read_arf() functions for reading X-ray spectral data files following NASA HEASARC standards. 3. Implement SDSS optical spectrum loader with log-wavelength to linear conversion and metadata extraction. 4. Write comprehensive unit tests using TDD approach to ensure robust and reliable code.
Jun 16 - Jul 12(Midterm / 2nd Eval)	1. OGIP parser fully migrated to Spectra.jl with OGIPSpectrum type working correctly for X-ray data. 2. SDSS data loading implemented and tested. Submit midterm evaluation and document progress.
Jul 12 - Aug 11(Coding Phase 2)	1. Implement HST UV spectrum loader and create unified load_spectrum() with automatic format detection. 2. Implement rebinning routines: minimum flux, signal-to-noise ratio, and count rates. 3. Add wavelength ↔ energy conversion and redshift correction (adjusting wavelength/energy shifts). 4. Write comprehensive tests and documentation with usage examples.
Aug 11 - Aug 25(Final Evaluation)	1. Final testing and submit revamped Spectra.jl with 3 file formats (OGIP, SDSS, HST). 2. Complete documentation with examples. Submit final code and mentor evaluations
Post-GSoC(If Timeline Allows)	1. Add support for additional file formats (JWST NIRSpec, IFU spectra, radio spectral cubes).

	2. Contribute to integration with other JuliaAstro packages like AstrolImages.jl and Cosmology.jl.
--	--

NOTE: Additionally, more work will be added, and it will be flexible dates. These are just for context :)

5.2 Availability

Since this is a 175-hour project, I plan to dedicate around 3–4 hours each weekday and 6-7 hours on weekends and holidays. As a student, I'll make sure to manage my time well to maintain a healthy balance between academics and GSoC. I'm genuinely excited about the project and fully committed to giving it my best.

6.0 Some of My Projects:

6.1 [Lung Cancer Detection using CNN](#): This project uses Convolutional Neural Networks to detect lung cancer from medical images. Built with Python and deep learning frameworks, it demonstrates my experience with scientific data analysis and image processing.

7.0 What I am studying and for what

[1] https://heasarc.gsfc.nasa.gov/docs/heasarc/ofwg/docs/spectra/ogip_92_007/ogip_92_007.html

From this document, I studied the OGIP standard for PHA (Pulse Height Analyzer) files - understanding the SPECTRUM extension and required columns like CHANNEL, COUNTS, QUALITY, GROUPING.

[2] https://heasarc.gsfc.nasa.gov/docs/heasarc/caldb/docs/memos/cal_gen_92_002/cal_gen_92_002.html

This HEASARC document explains RMF (Response Matrix File) and ARF (Ancillary Response File) formats. Essential for understanding how X-ray telescope responses are stored.

[3] <https://github.com/JuliaAstro/SpectralFitting.jl/blob/main/src/datasets/ogip.jl>

I am studying the existing OGIP parser implementation that needs to be migrated to Spectra.jl. Understanding the OGIPDataset struct and reading functions.

[4] <https://github.com/JuliaAstro/Spectra.jl/pull/24>

This PR discussion helped me understand the project direction and what changes are planned for Spectra.jl's architecture.

[5] <https://juliaastro.org/dev/Spectra.jl/stable/>

Studying the current Spectra.jl codebase, especially the AbstractSpectrum type hierarchy and how SingleSpectrum and EchelleSpectrum are implemented.

[6] https://www.sdss4.org/dr17/spectro/spectro_basics/

SDSS spectroscopy documentation. I studied how SDSS stores spectral data with loglam (log10 wavelength), flux, and ivar (inverse variance) columns. The logarithmic wavelength grid has spacing of $\log_{10} \lambda_{i+1} - \log_{10} \lambda_i = 0.0001$.

[7] <https://hst-docs.stsci.edu/cosdhub/chapter-2-cos-data-files/2-4-cos-data-products>

COS Data Handbook explaining x1d.fits file format. The extracted 1-D spectrum tables contain WAVELENGTH, FLUX, ERROR columns which I'll use for HST UV spectrum loader implementation.

[8] <https://juliaastro.org/dev/FITSIO.jl/stable/>

Learning FITSIO.jl for reading FITS files in Julia. Understanding how to read HDUs, headers, and table data.

[9] https://en.wikipedia.org/wiki/Astronomical_spectroscopy

Background reading on astronomical spectroscopy to understand different wavelength regimes (X-ray, UV, optical, infrared) and why each requires different data handling.

8.0 Appreciation:

First of all, I would like to express my sincere appreciation for the OpenAstronomy community. It truly feels right to be here—welcoming, insightful, and inspiring. I've already learned a great deal just by exploring the codebase and documentation.

The project resources including [PR #24](#) discussions helped me understand the project direction clearly and provided a well-structured foundation for my preparation.

I look forward to working closely with the mentors (@fjebaker, @icweaver, @cgarling, @barrettp) and contributing to the JuliaAstro ecosystem.

9.0 Gsoc:

Have you participated previously in GSoC? When? With which project?

Nope, this is my first GSOC

Are you also applying to other projects?

Yes , in one large project: (Integrate a modern lazy-loading alternative for large-scale spectroscopic database processing) if i got selected